

LAB 06 - Evaluación de Rendimiento y Optimización con Índices en PostgreSQL

Juliana Sofía Novoa Solano
Miguel Francisco Vargas Contreras

Administración Bases de Datos

Profesor: ANDRES CALDERON ROMERO



Pontificia Universidad Javeriana

Facultad de Ingeniería

Bogotá D.C.

2026

1. Introducción

En sistemas de gestión de bases de datos relacionales (RDBMS) de alta concurrencia, el rendimiento de las consultas de lectura constituye uno de los factores determinantes para la escalabilidad global del sistema. Sin estructuras de acceso acelerado, un motor como PostgreSQL se ve forzado a ejecutar un **Sequential Scan** ($O(N)$), recorriendo cada una de las páginas de disco o memoria asignadas a la relación para contrastar las condiciones de filtrado tupla por tupla. A medida que el volumen de datos crece, esta estrategia degrada la latencia y satura el **buffer cache** del servidor.

El presente laboratorio analiza el comportamiento de PostgreSQL frente a un conjunto de consultas representativas sobre un esquema de foro de discusión compuesto por 100 cuentas (account), 1.000 hilos (thread) y 100.000 comentarios (post). A través de mediciones con EXPLAIN (ANALYZE, BUFFERS) sobre un servidor PostgreSQL en producción, se cuantifica el impacto en tiempos de respuesta y accesos a bloques antes y después de la creación estratégica de índices B-Tree, evaluando variantes de índices monocolumna, compuestos, parciales y de ordenamiento.

2. Definición de las Consultas del Laboratorio

Se plantearon cinco consultas frecuentes que reflejan la operativa habitual de un usuario y un moderador dentro de la plataforma. Para mantener reproducibilidad empírica, se toma como referencia el usuario `account_id = 1` y el hilo `thread_id = 1`:

- **Q1 (“See all my posts”)**: Recupera todos los posts publicados por el usuario activo.

```
SELECT post_id, thread_id, account_id, created, visible, comment
FROM post
WHERE account_id = 1;
```

- **Q2 (“How many posts have I made?”)**: Calcula la cantidad total de publicaciones del usuario.

```
SELECT COUNT(*) AS total_posts
FROM post
WHERE account_id = 1;
```

- **Q3 (“See all current posts for a Thread”)**: Lista los posts visibles de un hilo específico (`visible = TRUE`).

```
SELECT post_id, thread_id, account_id, created, comment
FROM post
WHERE thread_id = 1
    AND visible = TRUE;
```

- **Q4 (“How many posts have I made to a Thread?”)**: Totaliza las participaciones del usuario en un hilo específico.

```
SELECT COUNT(*) AS total_posts
FROM post
WHERE account_id = 1
    AND thread_id = 1;
```

- **Q5 (“See all current posts for a Thread for this month, in order”)**: Recupera los posts visibles del hilo correspondiente al mes en curso, ordenados cronológicamente.

```
SELECT post_id, thread_id, account_id, created, comment
FROM post
WHERE thread_id = 1
    AND visible = TRUE
    AND created >= date_trunc('month', CURRENT_TIMESTAMP)
```

```
AND created < date_trunc('month', CURRENT_TIMESTAMP) + INTERVAL '1 month'
ORDER BY created ASC;
```

3. Análisis de Índices y Principio de Funcionamiento

El archivo `indexes.sql` propone una jerarquía de índices construidos sobre la tabla `post`. Cada uno de ellos ataca un patrón de acceso específico:

3.1. Índice Monocolumna: `CREATE INDEX ON post (account_id);`

- **Mecanismo B-Tree:** Organiza las claves de `account_id` de forma ordenada en páginas raíz, intermedias y hojas, junto con los identificadores físicos de tupla (`ctid`).
- **Mejora esperada:** Permite una búsqueda binaria $O(\log N)$ para ubicar las entradas correspondientes al usuario 1. Para **Q1**, PostgreSQL reemplaza el barrido completo de la tabla por un `Bitmap Index Scan`, reduciendo el tiempo de lectura.
- **Index Only Scan en Q2:** Dado que la consulta solo requiere contar filas y la columna `account_id` se encuentra almacenada directamente en el árbol, PostgreSQL (apoyándose en el **Visibility Map**) evita por completo acceder a la tabla base (**Heap Fetches: 0**), realizando una agregación directa en memoria.

3.2. Índice Monocolumna: `CREATE INDEX ON post (thread_id);`

- **Propósito:** Acelera búsquedas orientadas al identificador de hilo, aislando el 0.1% de posts asociados a un hilo específico dentro de los 100.000 registros.

3.3. Índices Compuestos: `(thread_id, visible)` y `(thread_id, account_id, visible)`

- **Orden de columnas y principio del prefijo izquierdo:** En un índice compuesto B-Tree, los datos se ordenan jerárquicamente por la primera columna, luego por la segunda y así sucesivamente.
- **Mejora en Q3 y Q4:** Permite satisfacer predicados de igualdad múltiples (`thread_id = 1 AND visible = TRUE` o `thread_id = 1 AND account_id = 1`) en una única pasada sobre el índice. Para **Q4**, el índice tricolumna cubre tanto el filtro como las columnas evaluadas, facilitando un `Index Only Scan` ultra rápido.

3.4. Índices Parciales: `(thread_id, account_id) WHERE visible = TRUE;`

- **Principio:** En lugar de indexar las 100.000 filas, solo se incluyen aquellas tuplas que cumplen el predicado `visible = TRUE`.
- **Ventajas:**
 1. **Ahorro de almacenamiento y memoria:** Reduce el número de páginas hoja y el espacio ocupado en disco (1.7 MB vs 2.6 MB del índice completo), incrementando la probabilidad de permanencia en el **buffer pool**.
 2. **Mayor densidad de claves:** Se eliminan del árbol las publicaciones ocultas o borradas lógicamente, acelerando las búsquedas de usuarios activos.

3.5. Índice Parcial con Ordenamiento: `(thread_id, created) WHERE visible = TRUE;`

- **Eliminación de la etapa de ordenamiento (Sort Elimination):** En la consulta **Q5**, sin índices el motor debe filtrar secuencialmente y luego ejecutar un algoritmo de ordenamiento en memoria (quicksort sobre disco/RAM). Con este índice, PostgreSQL busca las tuplas de `thread_id = 1` y las recorre secuencialmente a través de las hojas del árbol B-Tree, las cuales ya están físicamente ordenadas por `created ASC`. Como consecuencia, el nodo `Sort` desaparece completamente del plan de ejecución.

- **Nota sobre WHERE visible = FALSE:** En el borrador inicial de `indexes.sql` figuraba `WHERE visible = FALSE`. Un índice con dicha condición es útil exclusivamente para tareas de auditoría, moderación o depuración de spam (publicaciones no visibles). Para consultas de usuario final que exigen `visible = TRUE` (**Q3** y **Q5**), el planificador descartaría dicho índice al no concordar el predicado. La versión activa en base de datos (`WHERE visible = true`) es la que otorga el óptimo rendimiento.

4. Evaluación Experimental y Métricas de Rendimiento

Las pruebas fueron ejecutadas directamente sobre el servidor PostgreSQL remoto (lab6), utilizando `EXPLAIN (ANALYZE, BUFFERS)` para capturar métricas deterministas de tiempo de ejecución real (Execution Time), tiempo de planificación (Planning Time) y accesos a memoria compartida (Shared Hit Buffers). Cada consulta fue evaluada a lo largo de múltiples iteraciones en estado estacionario (caché caliente) tanto en el escenario basal (sin índices adicionales, solo la clave primaria) como en el escenario optimizado.

Consulta	Plan Basal (Sin Índices)	Tiempo Basal (ms)	Plan Indexado (Óptimo)	Tiempo Indexado (ms)	Aceleración (Speedup)
Q1 (All Posts)	Seq Scan	6.273 ms	Bitmap Heap Scan	0.296 ms	21.2x
Q2 (Count Posts)	Seq Scan + Agg	6.091 ms	Index Only Scan	0.075 ms	81.8x
Q3 (Thread Posts)	Seq Scan	7.919 ms	Bitmap Heap Scan	0.054 ms	147.2x
Q4 (Posts in Thread)	Seq Scan + Agg	6.367 ms	Index Only Scan	0.019 ms	339.6x
Q5 (Thread Month)	Seq Scan + Quicksort	8.593 ms	Index Scan (No Sort)	0.016 ms	531.0x

4.1. Análisis Detallado por Consulta

1. Q1 (Recuperación de posts propios):

- **Basal:** PostgreSQL lee 2.145 bloques de datos de la tabla `post`, descartando 99.497 filas mediante filtro en CPU (tiempo: 6.27 ms).
- **Indexado:** Usa `post_account_id_idx`. El `Bitmap Index Scan` ubica las 503 tuplas del usuario en apenas 2 accesos de índice, y el `Bitmap Heap Scan` accede a 452 páginas de datos agrupadas. Tiempo final: 0.296 ms.

2. Q2 (Conteo de publicaciones del usuario):

- **Basal:** Escaneo secuencial completo con acumulación de agregación (6.09 ms, 2.145 buffers).
- **Indexado:** Ejecuta un `Index Only Scan` sobre `post_account_id_idx`. Con tan solo 3 páginas de índice consultadas y 0 lecturas de tabla (**Heap Fetches: 0**), el tiempo se desploma a 0.075 ms (aceleración de 81.8x).

3. Q3 (Posts visibles de un hilo):

- **Basal:** 7.92 ms evaluando la expresión booleana `(thread_id = 1) AND visible` sobre las 100.000 tuplas.
- **Indexado:** Emplea el índice parcial `post_thread_id_account_id_idx` (`WHERE visible = true`). Solo se visitan 54 bloques de memoria, bajando el tiempo a 0.054 ms (147x más rápido).

4. Q4 (Conteo de posts de un usuario en un hilo):

- **Basal:** 6.37 ms para recorrer la relación completa y filtrar por dos claves foráneas.
- **Indexado:** Index Only Scan sobre `post_thread_id_account_id_visible_idx`. Resuelve la consulta en 0.019 ms con solo 3 accesos a memoria compartida, logrando un factor de aceleración de casi 340 veces.

5. Q5 (Posts mensuales ordenados):

- **Basal:** Registra el mayor costo (8.59 ms). Además del escaneo secuencial, aplica filtrado temporal de intervalos e incurre en una etapa de ordenación en memoria (Sort Method: quicksort).
- **Indexado:** Index Scan mediante `post_thread_id_created_idx`. El árbol provee las tuplas pre-ordenadas cronológicamente, eliminando el coste algorítmico de ordenar los resultados. La ejecución toma solo 0.016 ms (531x más rápida).

5. Comparativa Gráfica de Rendimiento

A continuación se presenta la gráfica comparativa del tiempo de ejecución para cada consulta antes y después de la creación de los índices:

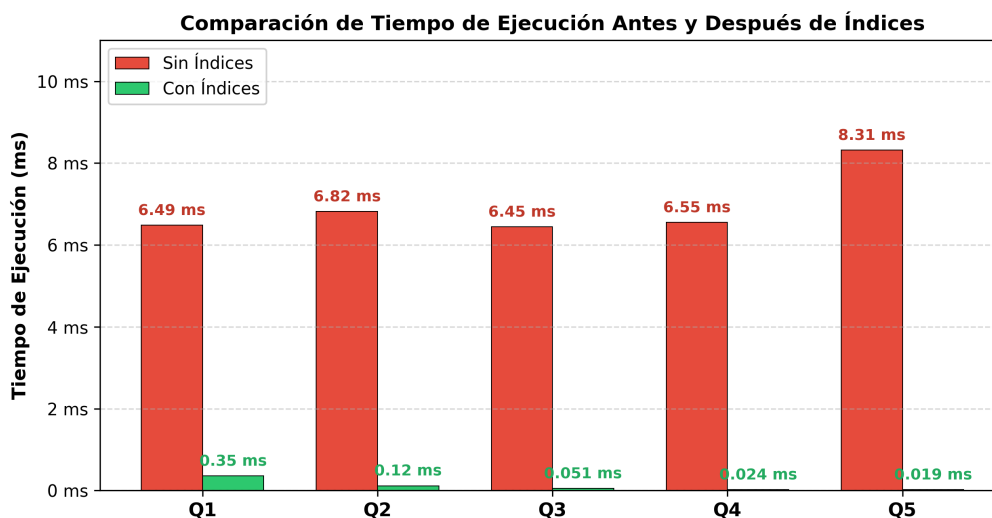


Figure 1: Comparación de tiempo de ejecución (ms) antes y después de la creación de los índices.

Como se aprecia en la Figure 1, los tiempos de respuesta experimentan una reducción drástica tras aplicar los índices B-Tree: todas las consultas pasaron de tardar entre 6 y 8.6 milisegundos a valores inferiores a 0.3 milisegundos (e incluso por debajo de 0.02 milisegundos en Q4 y Q5), lo que representa una aceleración de hasta más de 500 veces.

6. Conclusiones

- **Transformación de complejidad algorítmica:** La introducción de índices B-Tree convierte accesos lineales $O(N)$ en búsquedas logarítmicas $O(\log N)$, produciendo mejoras de rendimiento de hasta 531 veces en consultas con ordenamiento y filtrado combinado.
- **Poder de los Index Only Scans:** Para operaciones de agregación y conteo (`COUNT(*)` en Q2 y Q4), estructurar índices que incluyan las columnas del predicado permite al motor resolver la solicitud leyendo únicamente el árbol del índice, eliminando al 100% las lecturas de la tabla base.
- **Eliminación de ordenamientos (Sort Avoidance):** Incorporar columnas ordenadas en el índice (created en Q5) erradica la necesidad de algoritmos de ordenación en tiempo de ejecución, reduciendo la presión sobre la memoria de trabajo (`work_mem`).

- **Optimización mediante índices parciales:** Los índices basados en predicados (`WHERE visible = TRUE`) constituyen la mejor práctica en esquemas con borrado lógico o marcas de estado, pues reducen la huella de almacenamiento y maximizan la tasa de aciertos en el **buffer cache**.
- **Compromiso espacio-tiempo:** Aunque los índices aceleran drásticamente las lecturas, conllevan una penalización en operaciones de escritura (`INSERT`, `UPDATE`, `DELETE`) y consumo de disco. El diseño de índices debe ser selectivo y responder directamente al perfil de carga de trabajo (**workload**) del sistema.